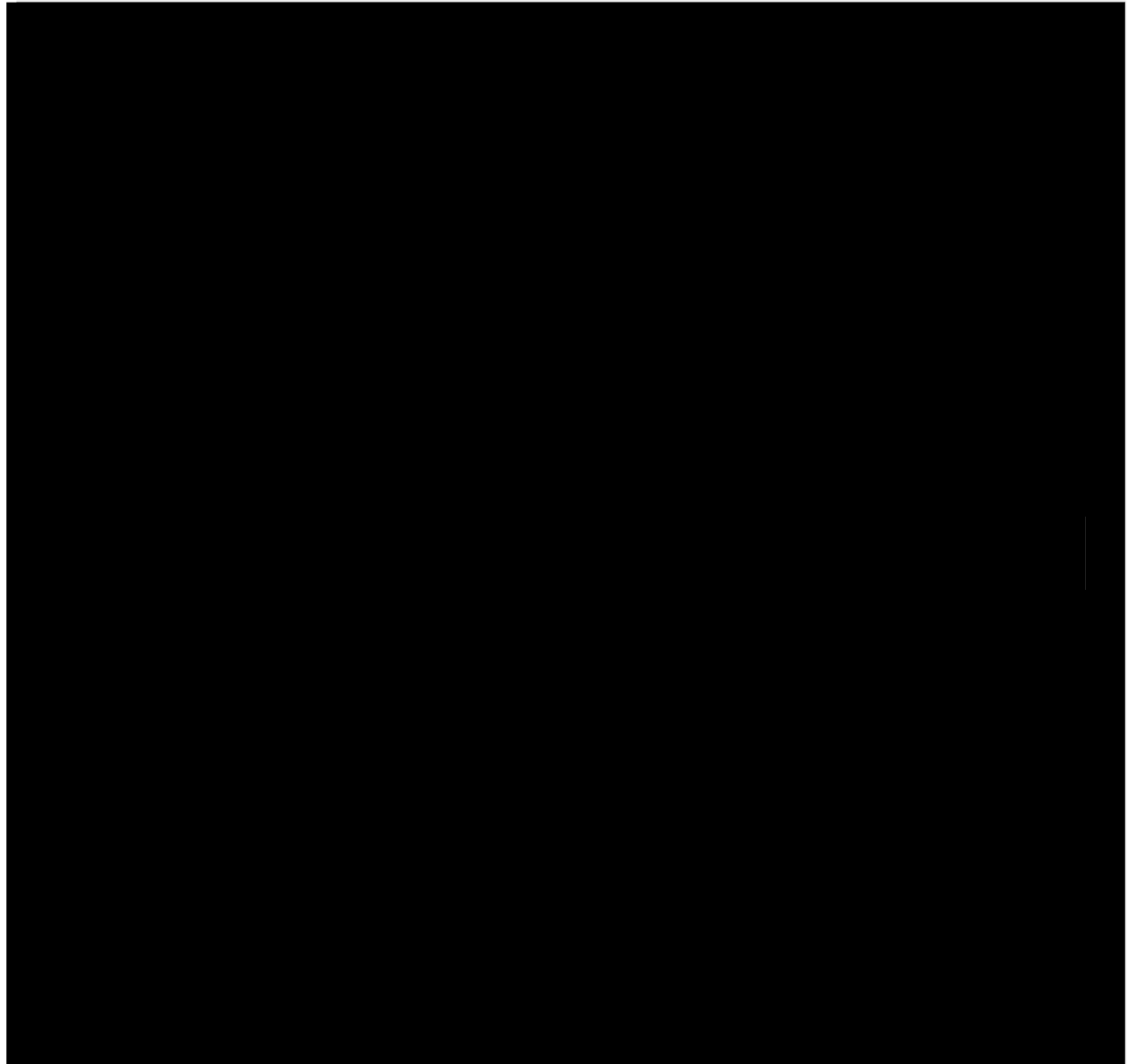


# Relazione progetto Algoritmi e Strutture Dati



Francesco Davini

## ► 1) Introduzione

- 1.1) Obiettivi del progetto
- 1.2) Analisi comparativa delle strutture dati
  - 1.2.1) Binary Search Tree (BST)
  - 1.2.2) Albero di Adelson-Velskij e Landis (AVL)
  - 1.2.3) Red-Black Tree (RBT)

## ► 2) L'esperimento

- 2.1) Descrizione codice esperimento
  - 2.1.1) La funzione sperimentale
  - 2.1.2) Insert/remove
  - 2.1.3) Progressione geometrica
  - 2.1.4) Test e misurazioni

## ► 3) Risultati e grafici

- 3.1) Linux (PC personale)
- 3.2) Windows con WSL (PC dell'università, processore i7-14700K)
- 3.3) Windows con WSL (PC personale)

## ► 4) Conclusioni

- 4.1) Interpretazione dei risultati
- 4.2) Esito finale dell'esperimento
- 4.3) Extra: analisi delle tempi in relazione alla cache

## 1) Introduzione

### 1.1) Obiettivi del progetto

Il progetto richiede l'implementazione e il confronto di 3 strutture dati:

- ▶ Alberi binari di ricerca semplici (BST)
- ▶ Alberi binari di ricerca AVL bilanciati
- ▶ Alberi di ricerca red-black (RBT)

Per ciascuna struttura dati elencata si vuole misurare il tempo mediano di inserimento di una nuova chiave quando l'albero contiene già  $n$  chiavi. Vogliamo ottenere stime robuste dei tempi mediani di esecuzione, per confrontare il comportamento osservato con la complessità teorica e verificare il comportamento delle 3 strutture dati.

### 1.2) Analisi comparativa delle strutture dati

Prima di iniziare necessario considerare alcuni fattori:

- ▶ Modalità di costruzione: Ogni struttura dati costruita deve mantenere la dimensione  $n$  richiesta.
- ▶ Affidabilità delle misure statistiche: Le misure sono basate su numerose iterazioni su alberi di forma diversa ma dimensione fissata pari a  $n$ .
- ▶ Ottimizzazione del costo di inizializzazione: per rendere l'esperimento eseguibile per diversi valori di  $n$  anche molto elevati.

Il tempo mediano per operazione  $\text{insert}(k)$  sarà indicato con  $T(n)$ . Un metodo adeguato per stimare tempo mediano deve soddisfare i seguenti criteri per ogni  $n$ :

- ▶ Dimensione controllata: Le operazioni d'inserimento vanno eseguite sugli alberi in cui ci sono esattamente  $n$  chiavi.
- ▶ Varietà di alberi: Le misurazioni devono essere effettuate su alberi distinti di dimensione  $n$ . Questi alberi devono essere costruiti in ordine casuale per avere delle stime di  $T(n)$  con distribuzione pseudo casuale.
- ▶ Robustezza statistica: Per ogni  $n$  serve un numero sufficiente di misure per calcolare la mediana riducendo così l'impatto del rumore e aumentando la precisione.
- ▶ Inizializzazione efficiente: Per ogni valore di  $n$  si genera un albero binario di ricerca di dimensione  $n$  una sola volta, per poi reimpiegare la struttura *-modificata da solo inserimenti e cancellazioni-* in tutte le misurazioni successive.

La consegna assegnata prevede la costruzione di un albero di dimensione  $n$  di ognuno dei 3 tipi, a cui segue l'applicazione di una serie di operazioni **insert/delete** che mantengono costante la dimensione, ma ne modificano continuamente la forma.

Per una struttura dati fissata tra le 3 elencate e un valore  $n$ , si eseguono i seguenti passaggi:

- ▶ Inizializzazione: In una fase preliminare non cronometrata, si costruisce **un insieme A con  $n+1$  chiavi**. Costruisco l'albero spostando  $n$  chiavi selezionate casualmente da A all'albero, ottenendo un albero con  $n$  chiavi e una forma iniziale casuale con una chiave inutilizzata.
- ▶ Batch delle misurazioni: Viene generata una lista di  $n+1$  numeri distinti, mescolata casualmente, i numeri partono da 0 e arrivano a  $n$ .
- ▶ I primi  $n$  elementi vengono inseriti nella struttura dati mentre uno solo rimane fuori.
- ▶ L'elemento che rimane fuori sarà il primo a essere inserito, subito dopo il primo elemento rimosso prenderà il suo posto, e così via.

Il procedimento descritto soddisfa tutti i 4 requisiti precedentemente elencati.

**È fondamentale** che il numero totale di chiavi non vari durante l'esperimento, ovviamente le chiavi da inserire non devono essere già presenti all'interno dell'albero, quelle da rimuovere devono invece essere presenti. Sia in inserimento che cancellazione la scelta dell'elemento su cui operare deve essere aleatoria. Constatiamo che è meglio evitare approcci per tentativi, il problema si riduce a mantenere due insiemi: le chiavi presenti nell'albero e quelle non presenti, aggiornati ad ogni inserimento o cancellazione (spiegato nel paragrafo successivo).

**La soluzione:** Rappresentare gli insiemi delle chiavi presenti e assenti tramite due array. La selezione di una chiave avviene generando un indice casuale e accedendo all'elemento nella posizione corrispondente ma questo è esattamente come si è poi agito.

**La soluzione migliore** (nonché quella utilizzata) sarebbe:

- ▶ Un array A di lunghezza  $n+1$ .
- ▶ Un intero  $m$  che delimita confine tra le chiavi presenti e quelle disponibili.
- ▶ Per inserire una chiave, è sufficiente selezionarla in  $A[m...n]$ .
- ▶ Per rimuovere una chiave, è sufficiente selezionarla in  $A[1...m]$ .

L'intervallo dei valori sul numero di nodi  $n$  considerati va da 1000 fino a un massimo di 10 000 000 con incrementi che seguono una serie geometrica, attraversando quindi valori nell'ordine delle migliaia e dei milioni. Questo ci permette di analizzare il comportamento delle strutture dati su diverse scale di grandezza, mantenendo un costo computazionale relativamente limitato.

Per ciascun valore di  $n$ , ogni struttura dati viene testata 100. I risultati possono variare non solo in base alla dimensione e alla forma dell'albero, ma anche a causa di fattori esterni al codice scritto come il sistema operativo, la cache e i processi in esecuzione in background. Per ottenere un valore rappresentativo, si utilizza la **mediana** che è un indicatore meno influenzato dai valori anomali rispetto ad esempio alla media. Questo processo viene ripetuto per ogni valore di  $n$  nell'intervallo considerato, e i valori ottenuti vengono riportati nel grafico.

### 1.2.1) Binary Search Tree (BST)

I Binary search tree si basano sulle seguenti regole:

- ▶ Per ogni nodo dell'albero, tutte le chiavi del sottoalbero sinistro sono minori o uguali alla chiave del nodo padre, mentre tutte quelle del sottoalbero destro sono maggiori o uguali.
- ▶ E una struttura ricorsiva, ogni sottoalbero sinistro o destro sarà a sua volta un BST.
- ▶ Di norma si usano chiavi uniche e distinte, ma ci sono anche maniere di gestire i duplicati (spostandoli a destra o a sinistra decidendo a priori) nel nostro caso utilizzeremo chiavi uniche non duplicate.

La caratteristica che distingue i BST dalle altre due strutture dati è l'assenza di qualsiasi tipo di bilanciamento implementato. L'albero quindi può trovarsi in situazioni diverse: può essere perfettamente bilanciato, mantenendo un'altezza di  $\log n$ , oppure, in casi estremi, estremamente sbilanciato con altezza  $n$ . In ogni caso, tutte le operazioni fondamentali su questo albero (ricerca, inserimento, cancellazione) hanno costo  $O(h)$ , dove  $h$  è l'altezza dell'albero.

Nei BST avremo l'**elemento massimo** nel nodo più a destra dell'albero e l'**elemento minimo** nel nodo più a sinistra. Un nodo **neo inserito** sarà sempre una foglia: si scorre ricorsivamente i sottoalberi a destra o sinistra fino a individuare la posizione corretta, con complessità  $O(n)$  nel caso peggiore (albero degenere) e  $O(\log n)$  nel caso medio (albero bilanciato).

Per la **cancellazione** non si presentano problemi se il nodo  $z$  da cancellare è una foglia. Se invece ha 1 o più figli non NIL, ci serve il **successore** di  $z$  (elemento minimo del sottoalbero destro di  $z$ ) che andrà a sostituirlo. Se il successore non è disponibile, uso il **predecessore** (elemento massimo del sottoalbero sinistro di  $z$ ). La cancellazione ha costo  $O(n)$  nel caso peggiore e  $O(\log n)$  nel caso bilanciato.

Per quella che è la consegna del nostro compito la struttura dati BST diventa la base per l'implementazione di tutte e due le altre strutture dati, questo implica anche l'implementazione nella classe BST di funzioni necessarie solo alle altre due strutture dati come ad esempio l'implementazione delle rotazioni a destra e sinistra.

### 1.2.2) Albero di Adelson-Velskij e Landis (AVL)

Gli AVL sono alberi binari di ricerca auto-bilanciati in cui per ogni nodo la differenza dell'altezza dei sottoalberi sinistro e destro ha valore compreso tra -1 e 1 (poi vedremo come questa differenza viene utilizzata).

Questo tipo di albero si distingue dal BST per la sua implementazione di un **meccanismo di bilanciamento** che va eseguito dopo le operazioni d'inserimento e cancellazione. Negli AVL, il bilanciamento consiste nel mantenere il valore del **balance factor (BF)** nell'intervallo  $[-1, 1]$  per ogni nodo presente nell'albero. Si distingue dai Red-Black tree per il meccanismo che utilizza per ottenere tale obiettivo.

Gli AVL utilizzano la funzione: **BF** =  $h(x.\text{left}) - h(x.\text{right})$  per calcolare il **balance factor**, dove  $h$  è la funzione che ritorna l'altezza. Come già definito, il balance factor deve rispettare l'intervallo  $[-1, 1]$ , così l'albero rimane bilanciato. In base al segno del valore capiremo se è sbilanciato sul sottoalbero destro o sinistro.

Se  $BF > 1$ , si verifica uno sbilanciamento che viene corretto tramite **rotazioni** destra o sinistra. In base alla forma dello sbilanciamento, l'albero può essere:

- ▶ Sbilanciato left-left: inserimento a sinistra con ulteriore figlio sinistro, si applica una rotazione a destra.
- ▶ Sbilanciato left-right: inserimento a sinistra con ulteriore figlio destro, si applica una rotazione sinistra sul figlio e una rotazione destra sul nodo.
- ▶ Sbilanciato right-right: inserimento a destra con ulteriore figlio destro, si applica una rotazione a sinistra.
- ▶ Sbilanciato right-left: inserimento a destra con ulteriore figlio sinistro, si applica una rotazione destra sul figlio e una rotazione sinistra sul nodo.

**La rotazione:** è una tecnica utilizzata nel ripristino del bilanciamento senza violare le proprietà dei BST:

- ▶ Rotazione a destra: il figlio sinistro di un nodo diventa nuovo padre del sottoalbero, mentre il nodo originale diventa figlio destro di quest'ultimo (esempio left-left).
- ▶ Rotazione a sinistra: il figlio destro di un nodo diventa nuovo padre del sottoalbero, mentre il nodo originale diventa il figlio sinistro (esempio right-right).

Negli AVL, gli **inserimenti** seguono la stessa logica dei BST. I nuovi nodi vengono sempre inseriti come foglie. Tuttavia a differenza di un BST classico, dopo ogni inserimento è necessario verificare la presenza di sbilanciamenti nell'albero, se necessario, ripristinare il bilanciamento tramite rotazioni.

Le **cancellazioni** richiedono particolare attenzione al problema del bilanciamento. Eliminare una foglia non presenta difficoltà, ma la rimozione di un nodo interno può rendere necessarie nuove rotazioni per ripristinare la proprietà di bilanciamento.

Grazie al mantenimento del bilanciamento, le **operazioni** di ricerca, inserimento e cancellazione hanno tutte complessità  $O(\log n)$  tuttavia come vedremo in seguito questo meccanismo di bilanciamento presenta anche dei difetti.

### 1.2.3) Red-Black Tree (RBT)

---

I Red-Black tree sono un altro tipo di albero bilanciato, che mantengono l'altezza minore possibile. Utilizzano la colorazione dei propri nodi (rossi o neri) per garantire che gli alberi rimangano bilanciati dopo inserimenti e cancellazioni.

Segue una serie di regole, rispettando comunque le 3 dei BST su unicità dei nodi, sottoalberi destri e sinistri e ricorsività della struttura dati, le regole sono:

- ▶ I nodi possono essere neri o rossi.
- ▶ La radice (root) dell'albero è sempre nera.
- ▶ Ogni nodo rosso ha due figli neri (valgono anche NIL).
- ▶ Per ogni nodo X, tutti i cammini verso le foglie contengono lo stesso numero di nodi neri.
- ▶ Questo numero di nodi neri trovati sul cammino è l'**altezza nera**, che deve sempre essere uguale per ogni cammino dalla radice alla foglia NIL.

Quando un nuovo nodo viene inserito in un RBT è di colore rosso (l'inserimento avviene secondo le regole dei BST). Si verificano poi le condizioni di bilanciamento.

Se ci sono violazioni si agisce con rotazioni e ricolorazioni:

- ▶ **Ricolorazioni:** modificano il colore di determinati nodi senza alterare la struttura dell'albero. Un caso classico si verifica quando il nodo inserito, il padre e lo zio sono tutti e tre rossi, nel qual caso il padre e lo zio vengono ricolorati di nero e il nonno dovrà diventare rosso. L'operazione può propagarsi ai livelli superiori dell'albero.
- ▶ **Rotazioni:** non sempre le ricolorazioni da sole bastano, bisogna quindi agire modificando la struttura dell'albero senza modificare le proprietà dei BST. Funzionano come negli AVL, ma nei RBT tendono a verificarsi più raramente.

I vari tipi di bilanciamento vengono gestiti a seconda delle casistiche:

- ▶ Lo zio è rosso: in questo caso il padre e lo zio del nodo neo inserito vengono ricolorati di nero, il nonno viene invece colorato di rosso, il controllo prosegue risalendo l'albero e il procedimento si può ripetere se risalendo l'albero si riscontra altre violazioni di questo tipo.
- ▶ Lo zio è nero o nullo e siamo in una situazione sbilanciata right-left o left-right: va eseguita una rotazione sul padre del nodo neo inserito.
- ▶ Lo zio è nero o nullo e siamo in una situazione sbilanciata right-right o left-left: va eseguita una rotazione sul nonno del nodo neo inserito, padre e nonno vanno infine ricolorati (se il nodo è rosso va ricolorato di nero e viceversa).

I RBT accettano un tipo di bilanciamento meno rigido rispetto alla struttura dati AVL, questo permette una gestione di inserimenti e cancellazioni che necessita di un numero mediamente inferiore di rotazioni, il tutto comunque mantenendo il costo delle operazioni di inserimento e cancellazione pari a  $O(\log(n))$

## 2) L'esperimento

### 2.1) Descrizione codice esperimento

Il codice implementa un ambiente per confrontare le prestazioni inerenti agli inserimenti nelle diverse strutture dati prima elencate, l'obiettivo come già detto è ottenere una stima robusta del tempo di inserimento di una chiave minimizzando il rumore e il costo di inizializzazione dell'esperimento.

La struttura si compone di:

- Funzione sperimentale: Il cuore pulsante dell'esperimento dove creiamo gli alberi da 0 per poi eseguire le operazioni richieste.
- Insert/remove: Prima della misurazione sono richieste cancellazioni e inserimenti per avere alberi casuali della stessa dimensione.
- Generazione dei valori di  $n$ : Per rispettare la progressione geometrica dei valori su cui agiamo (da 1000 a 10 000 000).
- Driver principale: Per l'esperimento e le misurazioni.

#### 2.1.1) La funzione sperimentale

```
def experiment_C(treeClass, n, insert_fn, remove_fn, L=100):
    # crea array con n+1 chiavi distinte per poi mischiarle
    random.shuffle(A)
    # chiavi fino a m-1 sono dentro mentre A[m] è fuori
    # costruisco l'albero una sola volta con n chiavi
    for i in range(m):
        insert_fn(tree, A[i])
    # vettore per salvare i tempi
    # inserisco chiave fuori da array e misuro il tempo
    k = A[m]
    start = time.perf_counter()
    insert_fn(tree, k)
    stop = time.perf_counter()
    times.append(stop - start)
    # provvisorio: l'albero ha n+1 chiavi
    # rimuovo una chiave e ritorno ad avere n nodi
    # swap
    # rimozione
    # torna a n chiavi
    node = tree.find(key_to_remove)
    if node is not None:
        # ritorno ad avere n chiavi e sono pronto alla prossima misurazione
        # ritorno la mediana dei tempi
```

La funzione `experiment_C` implementa il metodo dell'opzione C descritta dalla consegna. All'inizio creiamo una lista `A` di lunghezza  $n+1$  che viene mescolata con la funzione `shuffle` per ottenere una distribuzione casuale delle chiavi.

La variabile `m` rappresenta il confine tra le chiavi presenti e quelle temporaneamente escluse dall'albero. Nell'inizializzazione inseriamo nell'albero le prime  $n$  chiavi di `A` costruendo una struttura di dimensione fissata e forma casuale.

Successivamente eseguiamo `L` iterazioni sperimentali. Ad ogni iterazione:

- Si seleziona la chiave esterna all'albero.
- Viene misurato il tempo necessario all'inserimento con `time.perf_counter()`.
- La nuova chiave inserita porta temporaneamente la dimensione dell'albero a  $n + 1$ .
- Viene selezionata casualmente una chiave già presente nell'albero, che verrà poi rimossa (senza cronometrare l'operazione) per riportare l'albero a dimensione  $n$ .

In questo modo la dimensione della struttura dati rimane costante, mentre la sua forma varia continuamente, abbattendo così il costo di reinizializzazione di un nuovo albero.

Il garbage collector di python viene temporaneamente disabilitato per ridurre il rumore ed evitare eventuali operazioni automatiche di gestione della memoria che potrebbero interferire con il tempo rilevato durante una singola iterazione, al termine della quale verrà temporaneamente riattivato.

I tempi ottenuti vengono poi aggregati tramite la funzione `statistics.median()` per ottenere il valore mediano utilizzato perché è meno sensibile a valori outlier, dovuti a processi di sistema in corso cache o scheduling del sistema operativo.

### 2.1.2) Insert/remove

```
# insert e remove per ogni struttura dati
def bst_insert(tree, k): tree.insert(TreeNode(k))
def bst_remove(tree, node): tree.remove(node)
def avl_insert(tree, k): tree.insert(TreeNode(k))
def avl_remove(tree, node): tree.remove(node)
def rbt_insert(tree, k):
    node = TreeNode(k)
    tree.insert(node)
def rbt_remove(tree, node): tree.remove(node)
```

Le 3 strutture dati prese in considerazioni hanno metodologie differenti per la gestione di inserimenti e cancellazioni di nodi. Per uniformare il comportamento della funzione `experiment_C` abbiamo definito delle funzioni wrapper dedicate con il compito di:

- ▶ Creare correttamente i nodi.
- ▶ Richiamare le operazioni specifiche adatte alla struttura dati al momento in esame.
- ▶ Mantenere l'indipendenza di `experiment_C` dall'implementazione interna dei vari tipi di alberi (ad esempio per RBT dobbiamo colorare il nodo di rosso prima di inserirlo).

L'approccio utilizzato migliora la modularità del codice e consente di utilizzare `experiment_C` per tutte le strutture dati utilizzate senza modificarne la struttura.

### 2.1.3) Progressione geometrica

```
# progressione geometrica dei valori di n tra n_min e n_max
def get_n_values(n_min=1_000, n_max=10_000_000, count=100):
    c = (n_max / n_min) ** (1 / (count - 1))
    return sorted(set(int(n_min * (c ** i)) for i in range(count)))
```

I valori di  $n$  vengono generati tramite la progressione geometrica che parte da 1000 per arrivare a 10 000 000.

L'utilizzo della progressione geometrica (rispetto a quella lineare) ci permette di:

- ▶ Coprire intervalli dimensionali molto ampi.
- ▶ Osservare il comportamento asintotico delle strutture dati prese in esame.
- ▶ Distribuire i punti in maniera più uniforme su una scala logaritmica.

Si basa sulla costante moltiplicativa  $c = \left(\frac{n_{max}}{n_{min}}\right)^{\frac{1}{count-1}}$  e abbiamo la funzione `set()` che evita eventuali duplicati dovuti a arrotondamenti.

### 2.1.4) Test e misurazioni

```
results = []
n_values = get_n_values()
print(f"{'n':>10} | {'BST (μs)':>14} | {'AVL (μs)':>14} | {'RBT (μs)':>14}")
print("-" * 60)

results.append((n, times['BST'], times['AVL'], times['RBT']))
print(f"{'n':>10} | {times['BST']*1e6:>14.6f} | {times['AVL']*1e6:>14.6f} | {times['RBT']*1e6:>14.6f}")
```

La parte finale del codice contiene il driver principale dell'esperimento, responsabile di misurazioni e raccolta dei dati.

Prima delle misure viene eseguita una fase di warm-up per stabilizzare:

- ▶ Cache.
- ▶ Allocazioni dinamiche.
- ▶ Ottimizzazioni interne all'interprete di python.

Il warm-up è utile per ridurre l'impatto di questi fattori nelle prime esecuzioni.

Per ogni valore n generato dalla distribuzione geometrica:

- ▶ Genero in ordine casuale delle strutture dati.
- ▶ Vengono eseguiti in sequenza casuale i benchmark su BST, AVL e RBT.
- ▶ Vengono eseguite le misurazioni. I tempi mediani vengono poi salvati e stampati.

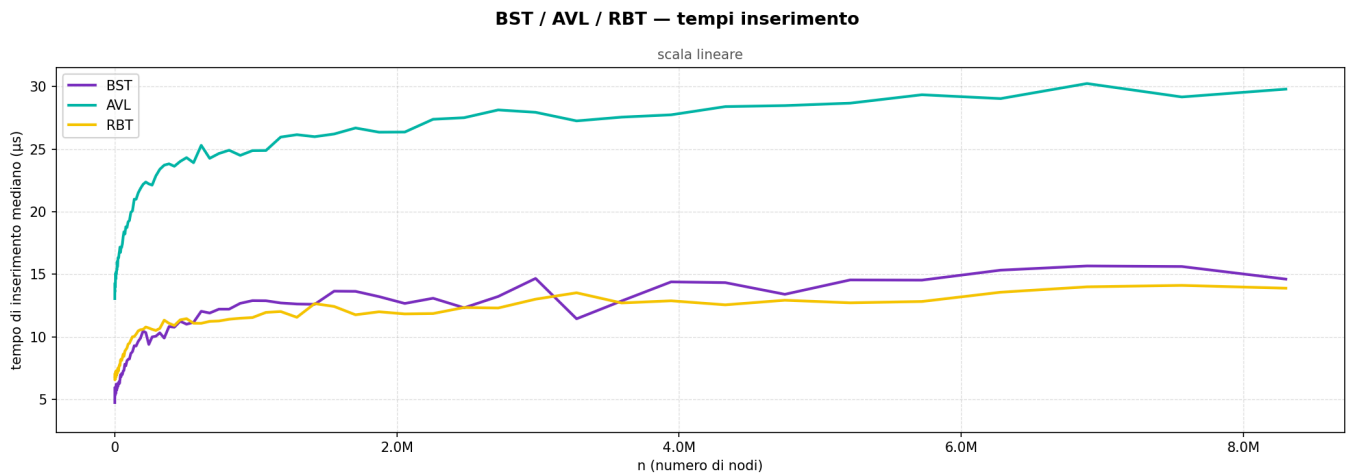
La riordinazione casuale della sequenza in cui testiamo le 3 strutture dati permette di evitare eventuali bias sistematici. Non ci resta che convertire i risultati in microsecondi per migliorare la leggibilità dell'output.

Secondo la consegna, il risultato dell'esperimento è correlato al sistema operativo presente sulla macchina su cui si esegue l'esperimento, motivo per cui abbiamo preso alcuni accorgimenti.

Per avere ulteriore conferma abbiamo quindi ripetuto l'esperimento su diverse macchine e con diversi sistemi operativi (utilizzando un PC di nostra proprietà e uno dell'università) per arrivare infine a 3 misurazioni, 2 con il PC di nostra proprietà (con sistemi operativi diversi) e uno con un PC del laboratorio dell'università.

### 3) Risultati e grafici

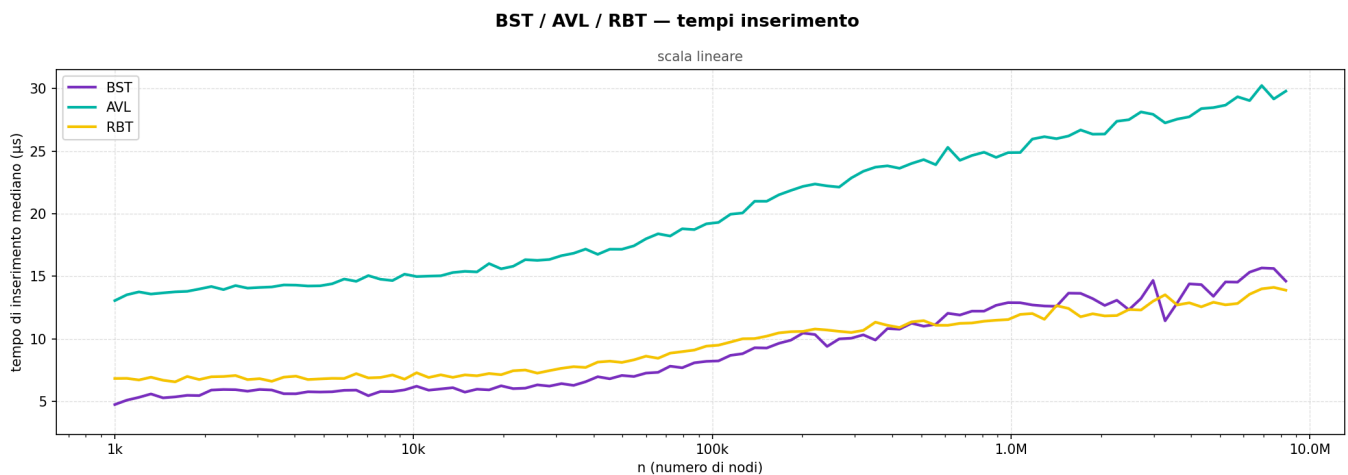
#### 3.1) Linux (PC personale)



Partiamo da questo grafico, realizzato sul PC personale con un processore [REDACTED] con il sistema operativo Linux.

Qui possiamo vedere i risultati in scala lineare, osserviamo come tutte e 3 le strutture dati testate, seppur con valori differenti, tendono ad un andamento logaritmico.

Questo è in linea con la premessa teorica sulla complessità per l'inserimento di nuove chiavi all'interno dei vari tipi di alberi.



Questo grafico in scala logaritmica ci permette di apprezzare meglio i dettagli relativi all'andamento dei dati dell'esperimento, soprattutto nei primi valori di n.



Come nell'altro grafico possiamo osservare che per ogni valore di  $n$  gli alberi AVL richiedono più tempo di BST e RBT per l'inserimento di una chiave, questo dovuto a un maggior numero di rotazioni necessarie al bilanciamento dell'albero.

Le rotazioni non sono infatti presenti nei BST, mentre nei RBT seppur presenti tendono a verificarsi più raramente, le numerose operazioni complesse di ribilanciamento rappresentano quindi uno svantaggio per gli AVL che vengono penalizzati dalle numerose rotazioni necessarie al bilanciamento dell'albero.

Concentrandoci su BST e RBT osserviamo un fenomeno interessante: i BST sono all'inizio più efficienti dei RBT, poco dopo  $n=100k$  i due grafici si incrociano e l'efficienza dei BST diminuisce.

Il fenomeno è dovuto al fatto che su un numero "piccolo" di nodi da gestire i RBT vengono, come gli AVL, penalizzati dalle rotazioni e operazioni necessarie al bilanciamento dell'albero.

I valori inerenti ai BST, seppur derivanti da una mediana di 100 test eseguiti, vicino  $n=100k$  sembrano risentire dell'assenza di bilanciamento della struttura.

Ne deduciamo quindi che nonostante l'implementazione del calcolo della mediana (per ridurre l'influenza di valori outlier [punto 1.2](#)) dopo una certa dimensione le operazioni di inserimento nei BST tendono ad avere un costo maggiore dovuto a un maggior tempo necessario a trovare la posizione dove inserire la nuova chiave.

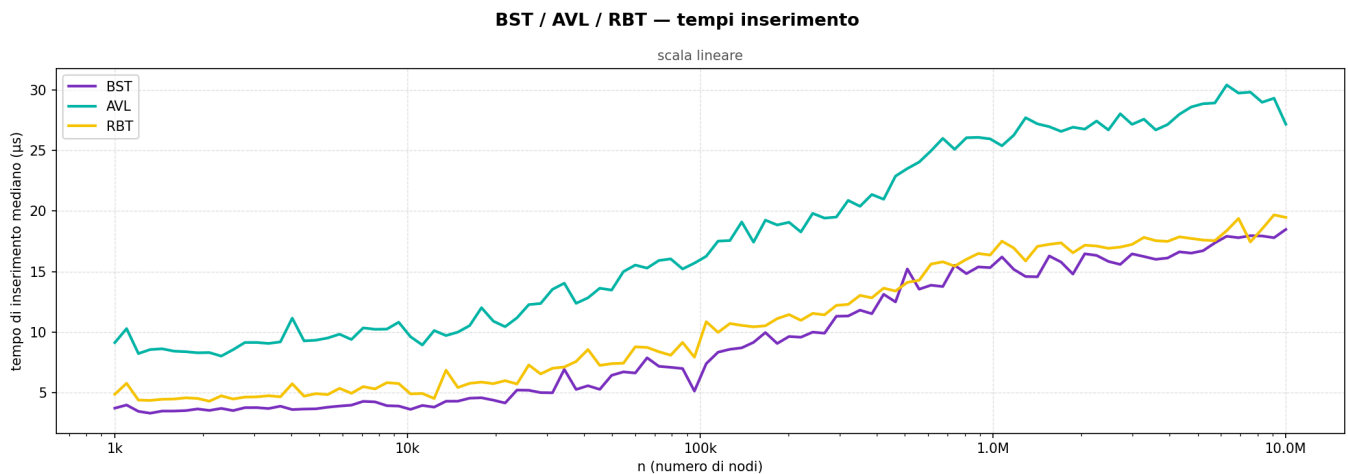
La dimensione dell'albero in cui si inserisce la nuova chiave è quindi un fattore fondamentale per i BST, nei RBT invece osserviamo un diverso andamento dei valori ottenuti e una crescita più uniforme.

Le ricolorazioni dei nodi e il numero inferiore di rotazioni necessarie rendono quindi nel lungo termine i RBT la struttura dati più efficiente tra le 3 su cui abbiamo eseguito i test.

Un altro fenomeno interessante che abbiamo osservato eseguendo l'esperimento su macchine differenti è stato il fatto che lo "scambio" tra i grafici inerenti BST e RBT avviene a valori di  $n$  diversi quando si opera su altre macchine.

### 3.2) Windows con WSL (PC dell'università, processore i7-14700K)

Visto quanto detto al punto precedente sul grafico in scala logaritmica, che mostra in maniera più chiara l'andamento dei dati, sia in questo che nel prossimo punto utilizzeremo solo il grafico in scala logaritmica.



L'andamento è simile a quello del grafico del punto precedente, tuttavia ci sono alcune questioni su cui vale la pena soffermarsi.

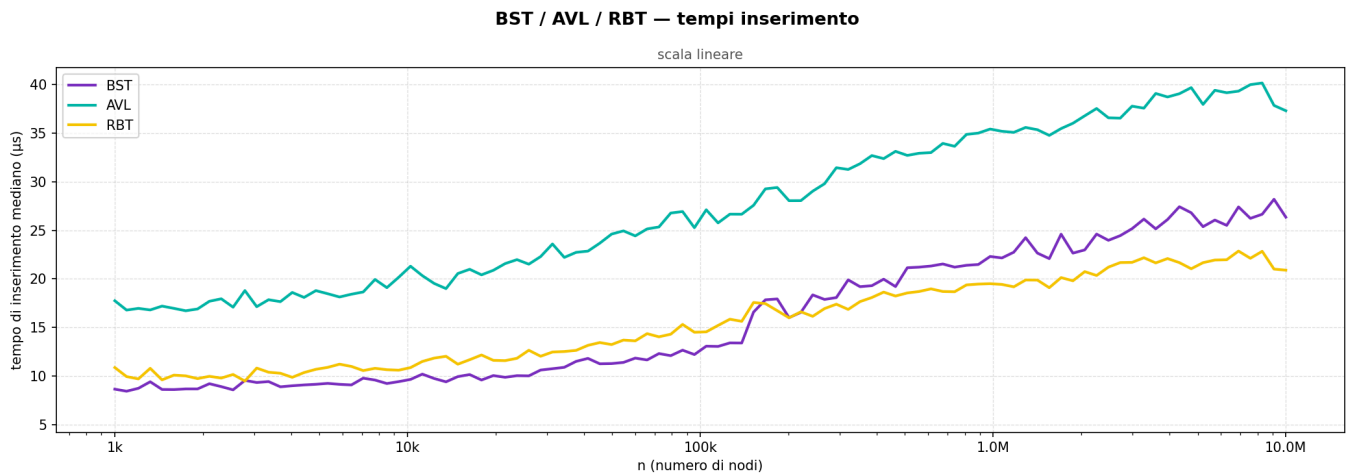
Osserviamo innanzi tutto un fenomeno comune a tutti i grafici esposti che abbiamo però meglio apprezzato dopo aver ripetuto il test su varie macchine, la presenza di spikes nei valori.

A seguito di varie ipotesi e analisi ne abbiamo concluso che sono dovute a transizioni di operazioni tra i vari livelli di memoria cache per poi arrivare alla RAM, le transizioni sono evidenziate dalle frecce che vedete sul grafico.

In questo grafico non abbiamo un incrocio netto tra i grafici di BST e RBT come nel punto precedente, questo perché la macchina utilizzata dispone di una quantità nettamente superiore di memoria cache che con la sua velocità riesce quindi a "ignorare" per più tempo i problemi prima citati sui BST come lo sbilanciamento eccessivo degli alberi su cui si eseguono i test.

Riusciamo ad apprezzare comunque un lento avvicinamento dei due grafici che specie verso le ultime misurazioni si incrociano 2 volte, non dubitiamo quindi del fatto che se il test si fosse protratto per altri valori più grandi alla fine avremmo avuto un incrocio netto come nel punto precedente.

### 3.3) Windows con WSL (PC personale)



Terminiamo quindi con il PC personale con il processore [REDACTED] e il sistema operativo Windows con WSL.

Qui possiamo tornare ad osservare l'incrocio simile tra i grafici RBT e BST di prima (le specifiche della macchina sono le stesse), tuttavia osserviamo un grafico più frastagliato.

Questa differenza rispetto al tentativo fatto con Linux e WSL è dovuta principalmente al numero di processi che i sistemi operativi Windows inseriscono automaticamente in background. Questo comportamento è stato estremamente evidente e accentuato durante i primi test, quando, per verificare se il codice funzionasse, lo si eseguiva mentre ci si dedicava ad altre attività sul PC.

## 4) Conclusioni

### 4.1) Interpretazione dei risultati

Partiamo dalla parte strettamente correlata alla consegna, i risultati sono complessivamente soddisfacenti.

Innanzitutto, osserviamo che tutti e tre i grafici, se visualizzati in scala lineare, rispettano l'andamento logaritmico previsto dalle premesse teoriche. Questo indica che le strutture dati sono correttamente implementate e che la funzione Experiment con wrapper, mediana e misurazioni hanno funzionato correttamente senza anomalie evidenti.

Passando poi alla parte che vediamo osservando i grafici in scala logaritmica osserviamo come gli AVL sono la struttura dati più lenta tra le 3 testate, questo perchè la loro necessità di continue rotazioni li porta ad avere frequenti operazioni anche complesse che influenzano il tempo di inserimento di nuovi nodi.

I RBT invece pur essendo all'inizio superati dai BST si sono rivelati la struttura dati più efficiente a lungo termine, osserviamo in tutti i test eseguiti come abbiano una crescita nettamente più omogenea delle altre 2 strutture dati.

Questo fattore è dovuto al bilanciamento attuato nei RBT che necessita di molte meno rotazioni rispetto agli alberi AVL. Questo significa quindi che necessita di meno tempo per inserire un nuovo nodo sia perchè essendo bilanciato ha meno sottoalberi in cui scendere sia perchè il bilanciamento non necessita di costanti operazioni costose.

I BST agiscono meglio di quanto ci si sarebbe aspettato nei primi valori di n, aiutato soprattutto dall'utilizzo della mediana per il ricavo di questi valori che ci porta dei risultati che somigliano a quelli che si avrebbero su un BST bilanciato.

Su macchine meno potenti, tuttavia, questo temporaneo vantaggio scompare non appena si supera una soglia che dipende dalle prestazioni della macchina utilizzata. Come già detto su un numero maggiore di nodi lo sbilanciamento degli alberi utilizzati inizia ad essere incisivo portando quindi a un incremento del tempo necessario all'inserimento di un nodo.

### 4.2) Esito finale dell'esperimento

L'esperimento ha dato dei risultati conformi a quelli attesi, tali risultati ci hanno permesso di trarre molte interessanti conclusioni sia sulle strutture dati su cui sono stati eseguiti i test che sulla gestione di tali strutture da parte della memoria delle macchine utilizzate (conclusioni citate al punto precedente).

Il codice oltre a passare i test preliminari presenti sulla piattaforma e-learning non presenta valori anomali, funziona correttamente e in accordo con la consegna che è stata seguita pedissecuamente, le osservazioni e le conclusioni tratte sono coerenti con i risultati ottenuti e verificate tramite controlli e ricerche.

### 4.3) Extra: analisi delle tempi in relazione alla cache

Un risultato che all'inizio ci è sembrato curioso è stato lo scambio tra RBT e BST: su alcune macchine i RBT inizialmente performavano peggio dei BST, ma dopo una certa soglia (rilevatasi consistete dopo diversi test) si invertivano, portando i RBT a completare le operazioni di inserimento più velocemente dei BST.

Analizzando i grafici e facendo esperimenti su più macchine e qualche ricerca abbiamo posto un'ipotesi (poi supportata da diversi test) secondo la quale gli aumenti di pendenza nel grafico sarebbero correlate al passaggio tra i vari livelli di cache per poi arrivare alla memoria RAM.

$$L1 \rightarrow L2 \rightarrow L3 \rightarrow RAM$$

Avendo eseguito l'esperimento su un pc che eseguiva Windows con WSL, del rumore di fondo è intrinseco nel metodo di rilevazione usato. Analizzando tuttavia lo spazio occupato dai singoli nodi degli alberi (come sono stati implementati da noi) e le dimensioni delle cache del processore, si rilevano dei nuovi pattern.

A seguito i dati relativi alla cache dei processori e alla dimensione dei nodi:

| Valore            |  | Intel Core i7-14700K | Struttura   | Peso nodo (bytes) |
|-------------------|--|----------------------|-------------|-------------------|
| L1 Cache          |  | 80 Kb X 8            | BST         | 584               |
| L2 Cache          |  | 2 Mb X 8             | AVL         | 664               |
| L3 Cache          |  | 33 MB                | RBT "red"   | 680               |
| Clock Speed/Turbo |  | 3.4 GHz /5.4 GHz     | RBT "black" | 672               |

Tenendo conto che questa stima sia una grossolana approssimazione (non viene tenuto conto di eventuali ottimizzazioni del compilatore, allineamenti di memoria, overhead di gestione della memoria e altri fattori), e che Python esegua il codice usando un solo thread (stimiamo quindi che venga eseguito su un solo core, e quindi abbia accesso solo ad una porzione della cache totale), possiamo stimare quando, per ciascun processore, si verifica il superamento progressivo del working set rispetto alla cache di un determinato livello e il passaggio al successivo.

Tutti i grafici a seguito riporteranno una scala logaritmica sia sul asse y che sull'asse x.

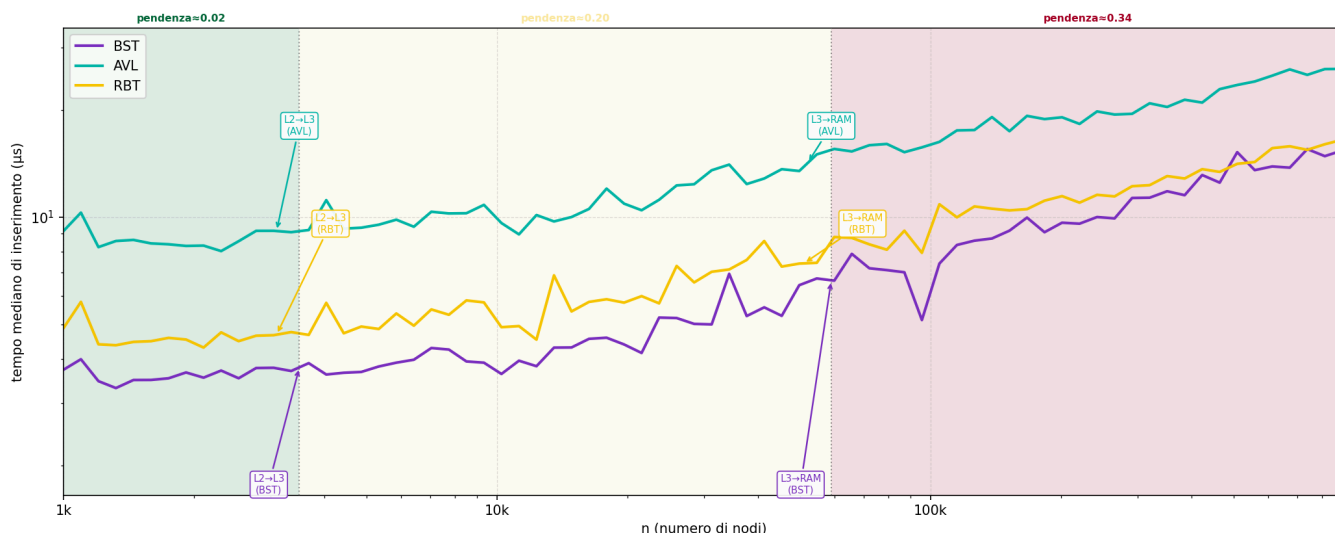
Vengono ora riportati i calcoli per il processore , con cache L1 da , L2 da e L3 da (tenere conto che nel grafico non sarà presente la L1 e L2 Cache perchè anche l'albero più piccolo supera la loro capacità massima):

| Struttura   | Peso Nodo (bytes) | Nodi per Cache L1 | Nodi per Cache L2 | Nodi per Cache L3 |
|-------------|-------------------|-------------------|-------------------|-------------------|
| BST         | 584               |                   |                   |                   |
| AVL         | 664               |                   |                   |                   |
| RBT "red"   | 680               |                   |                   |                   |
| RBT "black" | 672               |                   |                   |                   |

Viene quindi ripetuta l'operazione per il processore Intel Core i7-14700K, con Cache L1 da 80 KB, L2 da 2 MB e L3 da 33 MB:

| Struttura   | Peso Nodo (bytes) | Nodi per Cache L1   | Nodi per Cache L2    | Nodi per Cache L3      |
|-------------|-------------------|---------------------|----------------------|------------------------|
| BST         | 584               | 80 KB / 584 B ≈ 137 | 2 MB / 584 B ≈ 3,400 | 33 MB / 584 B ≈ 56,000 |
| AVL         | 664               | 80 KB / 664 B ≈ 121 | 2 MB / 664 B ≈ 3,000 | 33 MB / 664 B ≈ 50,000 |
| RBT "red"   | 680               | 80 KB / 680 B ≈ 118 | 2 MB / 680 B ≈ 2,900 | 33 MB / 680 B ≈ 48,000 |
| RBT "black" | 672               | 80 KB / 672 B ≈ 120 | 2 MB / 672 B ≈ 3,000 | 33 MB / 672 B ≈ 49,000 |

## BST / AVL / RBT — tempo di inserimento vs n - Intel i7-14700K



Notiamo un andamento simile a quello del [REDACTED], con una pendenza pressoché nulla quando si opera all'interno della L2 Cache, con un aumento nell'entrata in L3 e un aumento più significativo quando si supera la L3 e si accede alla RAM. Possiamo però notare come l'inclinazione in L3 sia maggiore rispetto a quella del [REDACTED], comportamento controintuitivo considerando che la L3 del processore Intel è maggiore rispetto a quella del [REDACTED] e le specifiche tecniche siano generalmente superiori.

Abbiamo ipotizzato che questo comportamento sia dovuto alla complessità dell'architettura del processore Intel e altri fattori cui esploreremo a breve.

Il processore Intel Core ha una architettura più complessa rispetto al [REDACTED], possedendo:

- ▶ 20 cores
  - 8 performance cores (P-cores)
    - L1 Cache: 80 KB per core (totale 640 KB)
    - L2 Cache: 2,048 KB per core (totale 16 MB)
  - 12 efficient cores (E-cores)
    - L1 Cache: 96 KB per core (totale 1,152 KB)
    - L2 Cache: 4,096 KB per cluster - ogni cluster ha 4 core (totale 12 MB)
- ▶ L3 Cache condivisa: 33 MB

Questa architettura genera una latenza relativamente più variabile. A questo aspetto si aggiunge il problema del clock speed dinamico: il processore intel avendo più cores, un consumo energetico complessivo più elevato (253 W) e un dissipatore stock, non è in grado di mantenere un clock speed Turbo costante, da qui la maggior quantità di spikes e l'aumento più significativo della latenza quando si accede alla L3 cache.

Di per contro il [REDACTED], avendo meno cores, un wattaggio più basso [REDACTED], un clock speed Turbo aumentato [REDACTED] gestito da un sistema di raffreddamento a liquido, riesce a mantenere un clock speed molto più stabile ed elevato, con una latenza più omogenea e prevedibile, con un incremento di latenza più graduale.

Testi supportata da diverse fonti, dove riportano una differenza significativa nei tempi di latenza:

| Cache             | [REDACTED] | Intel Core i7-14700K (ns) |
|-------------------|------------|---------------------------|
| L1 Cache          | [REDACTED] | 0.9                       |
| L2 Cache          | [REDACTED] | 3.6                       |
| L3 Cache          | [REDACTED] | 17.8                      |
| Clock Speed       | [REDACTED] | 5.4 GHz                   |
| Clock Speed Turbo | [REDACTED] | 5.6 GHz                   |

Le ipotesi trattate sembrerebbero supportate dal fatto che per alberi piccoli, l'intel ha un vantaggio significativo rispetto al [REDACTED] (5ns vs [REDACTED] per eseguire le operazioni sugli alberi) per poi convergere con alberi più grandi.

Le ipotesi quindi da noi formulata, dove si speculava che il tempo di inserimento non strettamente logaritmico fosse dovuto a un fenomeno più complesso del rumore di fondo del sistema operativo, sembrerebbe essere supportata da questi test e analisi effettuate. Rimane comunque un'ipotesi, vista le importanti approssimazioni fatte per il calcolo del passaggio tra i vari livelli di cache.

Questa è una analisi che non rientrava nei requisiti della consegna, ma abbiamo ritenuto interessante esplorare.